

One matrix

SPAR-PIBT against every alternative, on every map, in the one metric that decides.

Planner	AISLE-CONSTRAINED		OPEN FLOOR	
	BOTTLENECK	CORRIDORS	NARROW	MEDIUM
SPAR-PIBT (direction priced)	149	158	289	405
plain lifelong PIBT (no aisle layer)	118	131	354	502
same rule, enforced (direction deletes moves)	149	84	78	161
Token Passing (best of two variants)	8.5	0.0	not run	27
RHCR (windowed replanning)	12	0.5	not run	14
cost, ms per timestep — SPAR / PIBT / TP / RHCR	0.6 / 0.4 / 32 / 75	1.4 / 0.8 / 40 / 43	1.4 / 0.8 / — / —	2.0 / 1.0 / 191 / 98

Tasks delivered per 1000 timesteps; higher is better. best on that map significantly worse than SPAR-PIBT ($p \leq 0.05$). 5 seeds $\times$ 400 timesteps per cell, exact two-sided permutation test on the seed means. With 5 seeds a side the smallest attainable p is $2/252 \approx 0.008$, and a real difference can still land near 0.06.

WHAT THE MATRIX SAYS

1. **Against the published lifelong baselines, it is not close.** Token Passing and RHCR deliver 8–27 tasks per 1000 steps where SPAR-PIBT delivers 149–405, and they burn 20 $\times$ to 300 $\times$ more time per timestep doing it, and Token Passing delivers *nothing at all* on corridors ($p = 0.008$ everywhere). Plain PIBT beats them too: that gap belongs to PIBT, not to this work.

2. **Against plain PIBT, the map decides — and that split is the honest result.** The aisle layer is worth +27% and +21% where aisles actually constrain traffic ($p = 0.056, 0.063$ — suggestive, not yet significant at 5 seeds), and costs –18% and –19% on open floor ($p = 0.016, 0.008$ — significant). Traffic rules earn their keep in corridors and get in the way in a car park.

3. **Pricing direction instead of enforcing it is the largest effect measured anywhere in this system.** Same aisle logic, same signals, same everything — only the counterflow move goes from "costs 8" to "impossible": throughput falls by 1.9 $\times$ to 3.7 $\times$ ($p = 0.008$ on every map where an aisle ever commits a direction). On bottleneck no aisle is long enough to be managed, so the two are identical there — which is the control.

Why each row lands where it does

Token Passing / RHCR collapse because they plan paths, then hold them. Each robot reserves a full path through space-time and other robots must wait for it. In a narrow lifelong warehouse the reservations interlock: the first robot to stop is never asked to move aside, so a queue never resolves and the map deadlocks within tens of steps. PIBT's priority inheritance *pushes* the blocker instead of waiting for it, which is why it survives the same map at 30 $\times$ the throughput and a fraction of the cost.

SPAR wins on corridors and bottleneck because those maps have long single-file runs where two robots meeting head-on must have one of them reverse out. Committing a direction, admitting robots by capacity, and reversing the aisle when the other queue starves removes those meetings before they happen — head-on encounters drop by about a third on bottleneck.

SPAR loses on narrow and medium because those maps are mostly open floor. There is nearly always a way around, so a committed direction mostly lengthens routes, and the coordination it buys is coordination the map did not need. This is

reported at the same size as the wins; the method is a claim about constrained topologies, not a universal improvement.

Enforcing the same rule destroys it because PIBT's progress argument runs on free neighbours. Priority inheritance works by pushing a blocking robot into an adjacent cell; a one-way rule deletes roughly half of those cells in every corridor. Every robot then obeys the traffic scheme perfectly and the corridor fills up anyway. Priced instead, a step of progress is worth 10 and counterflow costs 8, so the escape move stays on the menu at a price a robot pays only when it is the last option left. That single sign — a term in the objective rather than a constraint on the domain — is what the method is named for.

Provenance

Every number is read from `docs/data/`, one committed dataset written by `experiments/run_all.py` (5 seeds, 400 timesteps, Poisson arrivals; bottleneck 16 robots at $\lambda=0.8$, corridors 35 at 1.0, narrow 30 at 1.2, medium 40 at 1.5). "SPAR-PIBT" is the fixed `aisle_direction_only` configuration on all four maps — the best SPAR configuration on each of them, so nothing here is picked per map. `tests/test_planner_docs.py` re-derives this table from the dataset and fails if the two disagree. The mechanism behind every row is in `spar-planner.pdf`; the interactive version of the same dataset is `docs/dashboard.html`.